

AI Assisted Coding

LAB ASSIGNMENT – 4.1

Name: N.Trinandini

Batch: 32

Hall Ticket No: 2303A52071

Q1. Zero-Shot Prompting (Basic Lab Task)

Task:

Write a Python function that classifies a given text as Spam or Not

Spam using zero-shot prompting.

Steps:

1. Construct a prompt without any examples.
2. Clearly specify the output labels.
3. Display only the predicted label.

Input:

"Congratulations! You have won a free lottery ticket."

Expected Output:

Spam

PROMPT:

```
20-01-2026 > assignment4.py > ...  
1 #Write a Python function that classifies a given text as Spam or Not Spam  
2
```

Code:

```
3 def classify_spam(text):  
4     spam_keywords = ["win", "free", "prize", "click", "buy now", "subscribe"]  
5     text_lower = text.lower()  
6     if any(keyword in text_lower for keyword in spam_keywords):  
7         return "Spam"  
8     else:  
9         return "Not Spam"  
10 # Function to classify text as Spam or Not Spam  
11 if __name__ == "__main__":  
12     user_input = input("Enter the text to classify: ")  
13     result = classify_spam(user_input)  
14     print(f"The text is classified as: {result}")
```

OUTPUT:

```
PS D:\college\3-2\AIAC\AIAC LAB> & "C:/Program Files/Python314/python.exe" "d:/college/3-2/AIAC/AIAC LAB/20-01-2026/assignment4.py"  
Enter the text to classify: what  
The text is classified as: Not Spam  
PS D:\college\3-2\AIAC\AIAC LAB> & "C:/Program Files/Python314/python.exe" "d:/college/3-2/AIAC/AIAC LAB/20-01-2026/assignment4.py"  
Enter the text to classify: free  
The text is classified as: Spam  
PS D:\college\3-2\AIAC\AIAC LAB> █
```

EXPLANATION:

Zero-shot prompting performs a task without providing any example inputs or outputs.

The prompt defines the task of classifying the text labels as Spam and Not Spam

Presence of keywords like *Congratulations*, *won*, *free*, and *lottery* leads to classification as Spam.

Q2. One-Shot Prompting (Emotion detection)

Task:

Write a Python program that detects the emotion of a sentence using one-shot prompting.

Emotions: ['happy', 'sad', 'angry', 'excited', 'nervous', 'neutral']

Steps:

1. Provide one labeled example inside the prompt.
2. Take a sentence as input.
3. Print the predicted emotion

PROMPT:

```
#Write a Python program that detects the emotion of a sentence, emotions: ['happy', 'sad', 'angry', 'excited', 'nervous', 'neutral']
```

CODE:

```
17
18 def detect_emotion(sentence):
19     emotion_keywords = {
20         "happy": ["joy", "pleased", "delighted", "content"],
21         "sad": ["unhappy", "sorrowful", "dejected", "downcast"],
22         "angry": ["mad", "furious", "irritate", "annoyed"],
23         "excited": ["thrilled", "elated", "eager", "enthusiastic"],
24         "nervous": ["anxious", "worried", "tense", "apprehensive"],
25         "neutral": []
26     }
27
28     sentence_lower = sentence.lower()
29     for emotion, keywords in emotion_keywords.items():
30         if any(keyword in sentence_lower for keyword in keywords):
31             return emotion.capitalize()
32     return "Neutral"
33
34 # Function to detect emotion in a sentence
35 if __name__ == "__main__":
36     user_sentence = input("Enter a sentence to detect emotion: ")
37     emotion = detect_emotion(user_sentence)
38     print(f"The detected emotion is: {emotion}")
```

OUTPUT:

```
PS D:\college\3-2\AIAC\AIAC LAB> & "C:/Program Files/Python314/python.exe" "d:/college/3-2/AIAC/AIAC LAB/20-01-2026/assignment4.py"
Enter a sentence to detect emotion: worry
Enter a sentence to detect emotion: worry
The detected emotion is: Neutral
The detected emotion is: Neutral
PS D:\college\3-2\AIAC\AIAC LAB> & "C:/Program Files/Python314/python.exe" "d:/college/3-2/AIAC/AIAC LAB/20-01-2026/assignment4.py"
Enter a sentence to detect emotion: joy
Enter a sentence to detect emotion: joy
The detected emotion is: Happy
```

Explanation:

The task of emotion detection is defined clearly with one instruction.

No multiple examples are provided, making it a one-shot experience.

The model identifies emotion using predefined keywords.

Q3. Few-Shot Prompting (Student Grading Based on Marks)

Task:

Write a Python program that predicts a student's grade based on marks using few-shot prompting.

Grades:

['A', 'B', 'C', 'D', 'F']

Grading Criteria (to be inferred from examples):

- 90–100 → A
- 80–89 → B
- 70–79 → C
- 60–69 → D
- Below 60 → F

PROMPT:

```
'''Write a python program that predicts student's grades based on marks
The grading criteria is as follows:
- 90-100: A
- 80-89: B
- 70-79: C
- 60-69: D
- Below 60: F
'''
```

CODE:

```
47 def predict_grade(marks):
48     """
49     This function predicts the student's grade based on the provided marks.
50
51     Parameters:
52     marks (int): The marks obtained by the student (0-100).
53
54     Returns:
55     str: The grade corresponding to the marks.
56     """
57     if 90 <= marks <= 100:
58         return "A"
59     elif 80 <= marks < 90:
60         return "B"
61     elif 70 <= marks < 80:
62         return "C"
63     elif 60 <= marks < 70:
64         return "D"
65     elif 0 <= marks < 60:
66         return "F"
67     else:
68         return "Invalid marks. Please enter a number between 0 and 100."
69 # Function to predict student's grade based on marks
70 if __name__ == "__main__":
71     try:
72         # Prompt the user to enter marks
73         user_marks = int(input("Enter the student's marks (0-100): "))
74
75         # Get the predicted grade
76         grade = predict_grade(user_marks)
77
78         # Display the predicted grade
79         print(f"The predicted grade for marks {user_marks} is: {grade}")
80     except ValueError:
81         print("Invalid input. Please enter an integer value for marks.")
82
```

OUTPUT:

```
PS D:\college\3-2\AIAC\AIAC LAB> "C:/Program Files/Python314/python.exe" "d:/college/3-2/AIAC/AIAC LAB/20-01-2026/assignment4.py"
Enter the student's marks (0-100): -98
The predicted grade for marks -98 is: Invalid marks. Please enter a number between 0 and 100.
PS D:\college\3-2\AIAC\AIAC LAB> "C:/Program Files/Python314/python.exe" "d:/college/3-2/AIAC/AIAC LAB/20-01-2026/assignment4.py"
Enter the student's marks (0-100): dgf
PS D:\college\3-2\AIAC\AIAC LAB> "C:/Program Files/Python314/python.exe" "d:/college/3-2/AIAC/AIAC LAB/20-01-2026/assignment4.py"
Enter the student's marks (0-100): dgf
Enter the student's marks (0-100): dgf
Invalid input. Please enter an integer value for marks.
PS D:\college\3-2\AIAC\AIAC LAB> "C:/Program Files/Python314/python.exe" "d:/college/3-2/AIAC/AIAC LAB/20-01-2026/assignment4.py"
Enter the student's marks (0-100): 98
The predicted grade for marks 98 is: A
The predicted grade for marks 98 is: A
```

Explanation:

Did with Few-shot prompting uses a small number of examples to help the model infer grading rules.

The model learns the mapping between marks and grades from the given examples.

Based on this learned pattern, it predicts the correct grade for new marks.

Q4. Multi-Shot Prompting (Indian Zodiac Sign Prediction using Month Name)

Task:

Write a Python program that predicts a person's Indian Zodiac sign (Rashi) based on the month of birth (month name) using multi-shot prompting.

Indian Zodiac Order (Simplified Month-Based Model): The Indian Zodiac cycle starts in March with Mesha and follows this order:

March → Mesha

April → Vrishabha

May → Mithuna

June → Karka

July → Simha

August → Kanya

September → Tula

October → Vrischika

November → Dhanu

December → Makara

January → Kumbha

February → Meena

PROMPT:

```
'''Write a python program to predict person Indian Zodiac sign based on month
The Zodiac signs are as follows:
March → Mesha
April → Vrishabha
May → Mithuna
June → Karka
July → Simha
August → Kanya
September → Tula
October → Vrischika
November → Dhanu
December → Makara
January → Kumbha
February → Meena'''
```

CODE:

```

98 def predict_zodiac_sign(month):
99     """
100     This function predicts the Indian Zodiac sign based on the given month.
101
102     Parameters:
103     month (str): The month for which to predict the Zodiac sign.
104
105     Returns:
106     str: The corresponding Zodiac sign.
107     """
108     zodiac_signs = {
109         "march": "Mesha",
110         "april": "Vrishabha",
111         "may": "Mithuna",
112         "june": "Karka",
113         "july": "Simha",
114         "august": "Kanya",
115         "september": "Tula",
116         "october": "Vrischika",
117         "november": "Dhanu",
118         "december": "Makara",
119         "january": "Kumbha",
120         "february": "Meena"
121     }
122
123     month_lower = month.lower()
124     return zodiac_signs.get(month_lower, "Invalid month. Please enter a valid month name.")
125 # Function to predict Indian Zodiac sign based on month
126 if __name__ == "__main__":
127     user_month = input("Enter the month to predict the Indian Zodiac sign: ")
128     zodiac_sign = predict_zodiac_sign(user_month)
129     print(f"The Indian Zodiac sign for {user_month} is: {zodiac_sign}")

```

OUTPUT:

```

PS D:\college\3-2\AIAC\AIAC LAB> & "C:/Program Files/Python314/python.exe" "d:/college/3-2/AIAC/AIAC LAB/20-01-2026/assignment4.py"
Enter the month to predict the Indian Zodiac sign: november
The Indian Zodiac sign for november is: Dhanu
PS D:\college\3-2\AIAC\AIAC LAB> & "C:/Program Files/Python314/python.exe" "d:/college/3-2/AIAC/AIAC LAB/20-01-2026/assignment4.py"
Enter the month to predict the Indian Zodiac sign: august
The Indian Zodiac sign for august is: Kanya

```

EXPLANATION:

Multi-shot prompting uses multiple examples to establish a clear pattern between month names and zodiac signs.

The model learns the complete month-to-Rashi mapping from the provided examples.

Using this learned pattern, it accurately predicts the Indian Zodiac sign for a given month.

Q5. Result Analysis Based on Marks

Task: Write a Python program that determines whether a student

Passes or Fails based on marks using Chain-of-Thought (CoT)

prompting.

Result Categories:

['Pass', 'Fail']

PROMPT:

```

132 #5
133 '''
134 Read the student marks from user
135 validate the marks within rang 0-100, if marks are outside range display invalid marks
136 if the marks are greater than or equal to 40 display pass
137 otherwise display fail
138 Handle non integer inputs gracefully'''
139

```

CODE:

```

140 def check_pass_fail():
141     """
142     This function checks whether a student has passed or failed based on their marks.
143
144     It prompts the user to input marks, validates the input,
145     and then determines if the student has passed or failed.
146
147     Returns:
148     None
149     """
150     try:
151         # Prompt the user to enter marks
152         marks = int(input("Enter the student's marks (0-100): "))
153
154         # Validate if marks are within the range 0-100
155         if marks < 0 or marks > 100:
156             print("Invalid marks! Please enter a number between 0 and 100.")
157             return
158
159         # Check if the student has passed or failed
160         if marks >= 40:
161             print(f"The student has Passed with {marks} marks.")
162         else:
163             print(f"The student has Failed with {marks} marks.")
164
165     except ValueError:
166         # Handle the case where the input is not a valid integer
167         print("Invalid input! Please enter a valid integer for marks.")
168 # Function to check if a student has passed or failed
169 if __name__ == "__main__":
170     check_pass_fail()

```

OUTPUT:

```

PS D:\college\3-2\AIAC\AIAC LAB> & "C:/Program Files/Python314/python.exe" "d:/college/3-2/AIAC/AIAC LAB/20-01-2026/assignment4.py"
Enter the student's marks (0-100): -8968
Invalid marks! Please enter a number between 0 and 100.
PS D:\college\3-2\AIAC\AIAC LAB> & "C:/Program Files/Python314/python.exe" "d:/college/3-2/AIAC/AIAC LAB/20-01-2026/assignment4.py"
Enter the student's marks (0-100): 78
The student has Passed with 78 marks.
PS D:\college\3-2\AIAC\AIAC LAB> & "C:/Program Files/Python314/python.exe" "d:/college/3-2/AIAC/AIAC LAB/20-01-2026/assignment4.py"
Enter the student's marks (0-100): wt
Invalid input! Please enter a valid integer for marks.

```

EXPLANATON:

Chain-of-Thought prompting breaks the decision into clear steps: input, validation, and result analysis.

The program first checks the validity of marks, then applies the pass/fail condition logically.

This step-by-step reasoning leads to the final result: Pass or Fail.

Q6 Voting Eligibility Check (Chain-of-Thought Prompting)

Task: Write a Python program that determines whether a person is eligible to vote using Chain-of-Thought (CoT) prompting.

PROMPT:

```

171
172 #6
173 '''Determine whether a person is eligible to vote,by reading age from user
174 | if age is 18 or more display Eligible
175 | otherwise display Not Eligible
176 | handle non integer and negative integer inputs gracefully and give output
177 '''

```

CODE:

```

178
179 def check_voting_eligibility():
180     """
181     This function checks whether a person is eligible to vote based on their age.
182
183     It prompts the user to input their age, validates the input,
184     and then determines if the person is eligible to vote.
185
186     Returns:
187     None
188     """
189     try:
190         # Prompt the user to enter their age
191         age = int(input("Enter your age: "))
192
193         # Validate if age is a non-negative integer
194         if age < 0:
195             print("Invalid age! Age cannot be negative.")
196             return
197
198         # Check if the person is eligible to vote
199         if age >= 18:
200             print("Eligible to vote.")
201         else:
202             print("Not Eligible to vote.")
203
204     except ValueError:
205         # Handle the case where the input is not a valid integer
206         print("Invalid input! Please enter a valid integer for age.")
207
208 # Function to check voting eligibility
209 if __name__ == "__main__":
210     check_voting_eligibility()

```

OUTPUT:

```

PS D:\college\3-2\AIAC\AIAC LAB> & "C:/Program Files/Python314/python.exe" "d:/college/3-2/AIAC/AIAC LAB/20-01-2026/assignment4.py"
Enter your age: -9868
Invalid age! Age cannot be negative.
PS D:\college\3-2\AIAC\AIAC LAB> & "C:/Program Files/Python314/python.exe" "d:/college/3-2/AIAC/AIAC LAB/20-01-2026/assignment4.py"
Enter your age: 89
Eligible to vote.
PS D:\college\3-2\AIAC\AIAC LAB> & "C:/Program Files/Python314/python.exe" "d:/college/3-2/AIAC/AIAC LAB/20-01-2026/assignment4.py"
Enter your age: wt
Invalid input! Please enter a valid integer for age.

```

EXPLANATION:

Chain-of-Thought prompting is applied by breaking the task into clear logical steps: input, validation, and decision.

The program first verifies that the entered age is valid before applying the eligibility rule.

Based on step-by-step reasoning, the final output is Eligible to vote or Not Eligible to vote.

Q7 Prompt Chaining (String Processing – Palindrome Names)

Task: Write a Python program that uses the prompt chaining technique to identify palindrome names from a list of student names.

PROMPT:

```

11
12 #7
13 '''Generate a list of student names
14 Traverse the list and generate the reverse of each name
15 Compare the original name with the reversed name and determine if it is a palindrome
16 if both are same display it is a palindrome
17 | otherwise display it is not a palindrome
18 '''
19
20

```

CODE:

```

219
220 def check_palindromes(student_names):
221     """
222     This function checks each name in the list to determine if it is a palindrome.
223
224     Parameters:
225     student_names (list): A list of student names.
226
227     Returns:
228     None
229     """
230     for name in student_names:
231         # Generate the reverse of the name
232         reversed_name = name[::-1]
233
234         # Compare the original name with the reversed name
235         if name.lower() == reversed_name.lower():
236             print(f"{name} is a palindrome.")
237         else:
238             print(f"{name} is not a palindrome.")
239 # List of student names
240 students = ["Anna", "Bob", "Cathy", "David", "Eve",
241            |   "Hannah", "Ian", "Jack", "Kayak", "Liam"]
242 # Check for palindromes in the list of student names
243 check_palindromes(students)
244

```

OUTPUT:

```

Invalid Input: Please enter a valid integer for age.
PS D:\college\3-2\AIAC\AIAC LAB> & "C:/Program Files/Python314/python.exe" "d:/college/3-2/AIAC/AIAC LAB/20-01-2026/assignment4.py"
Anna is a palindrome.
Bob is a palindrome.
Cathy is not a palindrome.
David is not a palindrome.
Eve is a palindrome.
Hannah is a palindrome.
Ian is not a palindrome.
Jack is not a palindrome.
Kayak is a palindrome.
Liam is not a palindrome.

```

EXPLANATION:

Prompt chaining divides the task into steps: generating names, reversing each name, and comparing results.

Each step uses the output of the previous step to continue processing.

This chained process identifies which student names are palindromes

Q8 Prompt Chaining (String Processing – Word Length Analysis)

Task: Write a Python program that uses prompt chaining to analyze a list of words. In the first prompt, generate a list of words. In the second prompt, traverse the list and calculate the length of each word. In the third prompt, use the output of the previous step to determine whether each word is Short (length less than 5) or Long (length greater than or equal to 5), and display the result for each word

PROMPT:

```

246 #8
247 '''
248 Generate a list of names
249 display only the names that are palindromes
250 calculate the length of each word which are palindromes
251 and display the length along with the name
252 Short if length is less than 5
253 Long if length is greater than or equal to 5
254 '''

```

CODE:


```

256 def display_palindrome_names(names):
257     """
258     This function displays the names that are palindromes along with their lengths.
259
260     Parameters:
261     names (list): A list of names.
262
263     Returns:
264     None
265     """
266     for name in names:
267         # Generate the reverse of the name
268         reversed_name = name[::-1]
269
270         # Check if the name is a palindrome
271         if name.lower() == reversed_name.lower():
272             length = len(name)
273             length_type = "Short" if length < 5 else "Long"
274             print(f"{name} is a palindrome. Length: {length} ({length_type})")
275
276 # List of names
277 name_list = ["Anna", "Bob", "Cathy", "David", "Eve",
278             "Hannah", "Ian", "Jack", "Kayak", "Liam"]
279 # Display palindrome names along with their lengths
280 display_palindrome_names(name_list)

```

OUTPUT:

```

PS D:\college\3-2\AIAC\AIAC LAB> & "C:/Program Files/Python314/python.exe" "d:/college/3-2/AIAC/AIAC LAB/20-01-2026/assignment4.py"
Liam is not a palindrome.
Anna is a palindrome. Length: 4 (Short)
Bob is a palindrome. Length: 3 (Short)
Eve is a palindrome. Length: 3 (Short)
Hannah is a palindrome. Length: 6 (Long)
Kayak is a palindrome. Length: 5 (Long)
PS D:\college\3-2\AIAC\AIAC LAB>

```

EXPLANATION:

Prompt chaining is used by breaking the task into stages: generating a list of words, computing their lengths, and classifying them as Short or Long.

Each stage uses the output from the previous step to continue the analysis.

This step-by-step chaining produces the final word length classification for each word.